ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 16

WebSocket + Push

Рассмотрим использование WebSocket (через библиотеку Socket.io) для двусторонней связи в реальном времени, а также реализацию push-уведомления, чтобы пользователи получали информацию даже при закрытом приложении. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Введение

`WebSocket` - это протокол, обеспечивающий постоянное двустороннее соединение между клиентом и сервером. В отличие от HTTP, где клиент всегда инициирует запрос, WebSocket позволяет серверу отправлять данные клиенту в любой момент без дополнительного запроса. Это делает его идеальным для чатов, лент активности, онлайн-игр и других приложений реального времени.

`Socket.io` - это JavaScript-библиотека для веб-приложений и обмена данными в реальном времени. Она обеспечивает двустороннюю связь между клиентом и сервером: как клиент, так и сервер могут инициировать отправку данных и получать данные от другой стороны без необходимости постоянно запрашивать сервер для обновлений страницы.

Библиотека состоит из двух частей: клиентской, которая запускается в браузере, и серверной для Node.js. Главным образом использует протокол WebSocket. Коммуникация между клиентской и серверной частями осуществляется посредствам передачи событий.

Клиент или сервер могут генерировать событие и отправлять его с сопутствующими данными. Например, клиент может отправить событие «message» с текстовым

содержимым сообщения, а сервер - событие «notification» с информацией о новом уведомлении.

`Push-уведомления` позволяют отправлять сообщения даже тогда, когда приложение не открыто в браузере.

В рамках данного практического занятия объединим эти подходы: через WebSocket будем получать события в реальном времени, а через Push - направлять напоминания пользователю.

Шаг 1. Подготовка сервера

Для работы реализации примера нам понадобится простой Node.js-сервер. Будем использовать:

- `express` – для раздачи статики и обработки HTTP-запросов.
- `socket.io` – для работы с WebSocket (удобный API, автоматическое переподключение).
- `web-push` – для отправки push-уведомлений.
- `body-parser` – для парсинга JSON в теле запросов;
- `cors` – для разрешения кросс-доменных запросов.

Выполните в папке проекта (или создайте отдельную папку server) инициализацию и установку:

```
npm init -y
npm install express socket.io web-push body-parser cors
```

1.1. Генерация VAPID-ключей

VAPID-ключи необходимы для идентификации вашего сервера при отправке push-уведомлений. Сгенерируйте их с помощью команды:

```
npx web-push generate-vapid-keys
```

Вы увидите вывод, похожий на:

```
Public Key:
BG...
Private Key:
...
```

Сохраните оба ключа – они понадобятся в коде сервера и клиента (публичный ключ).

1.2. Создание файла `server.js`

Теперь создадим файл `server.js` в корне проекта (или в папке сервера). Этот файл будет:

- раздавать статические файлы нашего приложения (HTML, JS, CSS, иконки);
- обрабатывать WebSocket-соединения;
- предоставлять эндпоинты для сохранения и удаления push-подписок;
- при получении события `newTask` от клиента рассылать его через WebSocket и инициировать отправку push-уведомлений всем подписанным пользователям.

```
const express = require('express');
const http = require('http');
const socketIo = require('socket.io');
const webpush = require('web-push');
const bodyParser = require('body-parser');
const cors = require('cors');
const path = require('path');

const vapidKeys = {
    publicKey: 'сюда публичный ключ',
    privateKey: 'сюда приватный ключ'
};

webpush.setVapidDetails(
    'mailto:your-email@example.com', // укажите свой email
    vapidKeys.publicKey,
    vapidKeys.privateKey
);

const app = express();
app.use(cors());
app.use(bodyParser.json());

app.use(express.static(path.join(__dirname, './'))); // если server.js в
корне

// Хранилище подписок
let subscriptions = [];

const server = http.createServer(app);
const io = socketIo(server, {
    cors: { origin: "*", methods: ["GET", "POST"] }
});

io.on('connection', (socket) => {
    console.log('Клиент подключён:', socket.id);

    // Обработка события 'newTask' от клиента
    socket.on('newTask', (task) => {
```

```
        // Рассылаем событие всем подключённым клиентам, включая отправителя
        io.emit('taskAdded', task);

        // Формируем payload для push-уведомления
        const payload = JSON.stringify({
            title: 'Новая задача',
            body: task.text
        });

        // Отправляем уведомление всем подписанным клиентам
        subscriptions.forEach(sub => {
            webpush.sendNotification(sub, payload).catch(err =>
console.error('Push error:', err));
        });
    });

    socket.on('disconnect', () => {
        console.log('Клиент отключён:', socket.id);
    });
});

// Эндпоинты для управления push-подписками
app.post('/subscribe', (req, res) => {
    subscriptions.push(req.body);
    res.status(201).json({ message: 'Подписка сохранена' });
});

app.post('/unsubscribe', (req, res) => {
    const { endpoint } = req.body;
    subscriptions = subscriptions.filter(sub => sub.endpoint !== endpoint);
    res.status(200).json({ message: 'Подписка удалена' });
});

const PORT = 3001;
server.listen(PORT, () => {
    console.log(`Сервер запущен на http://localhost:${PORT}`);
});
```

Шаг 2. Интеграция WebSocket и push на клиенте

Клиентская часть нашего приложения (файлы `index.html`, `app.js`, `sw.js`) уже содержит основу из предыдущих практик. Теперь мы добавим в неё поддержку Socket.IO и механизм подписки на push.

2.1. Подключение Socket.IO в `index.html`

В файле `index.html` перед закрывающим тегом `</body>` добавьте ссылку на клиентскую библиотеку Socket.IO:

```
<script src="https://cdn.socket.io/4.5.4/socket.io.min.js"></script>
<script src="app.js"></script>
```

2.2. Доработка `app.js`

В начало файла `app.js` (после получения ссылок на элементы) добавьте подключение к серверу:

```
const socket = io('http://localhost:3001');
```

2.2.1. Функции для работы с push-подписками

Добавим вспомогательную функцию для преобразования публичного VAPID-ключа из формата base64 в формат, понятный браузеру:

```
function urlBase64ToUint8Array(base64String) {
    const padding = '='.repeat((4 - base64String.length % 4) % 4);
    const base64 = (base64String + padding).replace(/\-/g,
'+').replace(/_/g, '/');
    const rawData = window.atob(base64);
    const outputArray = new Uint8Array(rawData.length);
    for (let i = 0; i < rawData.length; ++i) {
        outputArray[i] = rawData.charCodeAt(i);
    }
    return outputArray;
}
```

Теперь реализуем асинхронные функции `subscribeToPush` и `unsubscribeFromPush`:

```
async function subscribeToPush() {
    if (!('serviceWorker' in navigator) || !('PushManager' in window))
return;
    try {
        const registration = await navigator.serviceWorker.ready;
        const subscription = await registration.pushManager.subscribe({
            userVisibleOnly: true,
            applicationServerKey:
urlBase64ToUint8Array('ВАШ_ПУБЛИЧНЫЙ_VAPID_КЛЮЧ')
        });
        await fetch('http://localhost:3001/subscribe', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(subscription)
        });
        console.log('Подписка на push отправлена');
    } catch (err) {
```

```
        console.error('Ошибка подписки на push:', err);
    }
}

async function unsubscribeFromPush() {
    if (!('serviceWorker' in navigator) || !('PushManager' in window))
return;
    const registration = await navigator.serviceWorker.ready;
    const subscription = await registration.pushManager.getSubscription();
    if (subscription) {
        await fetch('http://localhost:3001/unsubscribe', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ endpoint: subscription.endpoint })
        });
        await subscription.unsubscribe();
        console.log('Отписка выполнена');
    }
}
```

2.2.2. Интерфейс для включения/отключения уведомлений

Добавим в `index.html` кнопки (они уже есть в предоставленном шаблоне):

```
<footer class="is-center">
    <button id="enable-push" class="button success">Включить
уведомления</button>
    <button id="disable-push" class="button error"
style="display:none;">Отключить уведомления</button>
</footer>
```

Теперь в `app.js` внутри блока регистрации Service Worker добавим логику управления этими кнопками и вызов функций подписки/отписки:

```
if ('serviceWorker' in navigator) {
    window.addEventListener('load', async () => {
        try {
            const reg = await navigator.serviceWorker.register('/sw.js');
            console.log('SW registered');

            const enableBtn = document.getElementById('enable-push');
            const disableBtn = document.getElementById('disable-push');

            if (enableBtn && disableBtn) {
                const subscription = await
reg.pushManager.getSubscription();
                if (subscription) {
                    enableBtn.style.display = 'none';
```

```
                    disableBtn.style.display = 'inline-block';
                }

                enableBtn.addEventListener('click', async () => {
                    if (Notification.permission === 'denied') {
                        alert('Уведомления запрещены. Разрешите их в
настройках браузера.');
                        return;
                    }
                    if (Notification.permission === 'default') {
                        const permission = await
Notification.requestPermission();
                        if (permission !== 'granted') {
                            alert('Необходимо разрешить уведомления.');
                            return;
                        }
                    }
                    await subscribeToPush();
                    enableBtn.style.display = 'none';
                    disableBtn.style.display = 'inline-block';
                });

                disableBtn.addEventListener('click', async () => {
                    await unsubscribeFromPush();
                    disableBtn.style.display = 'none';
                    enableBtn.style.display = 'inline-block';
                });
            }
        } catch (err) {
            console.log('SW registration failed:', err);
        }
    });
}
```

2.2.3. Отправка события при добавлении задачи

В функции `addNote` (которая вызывается при отправке формы) после сохранения задачи в `localStorage` добавим отправку события через WebSocket:

```
function addNote(text, datetime) {
    const notes = JSON.parse(localStorage.getItem('notes') || '[]');
    const newNote = { id: Date.now(), text, datetime: datetime || '' };
    notes.push(newNote);
    localStorage.setItem('notes', JSON.stringify(notes));
    loadNotes();

    // Отправляем событие на сервер
```

```
    socket.emit('newTask', { text, timestamp: Date.now() });
}
```

2.2.4. Получение события от других клиентов

Добавим обработчик, который будет показывать всплывающее сообщение при получении события `taskAdded` от сервера:

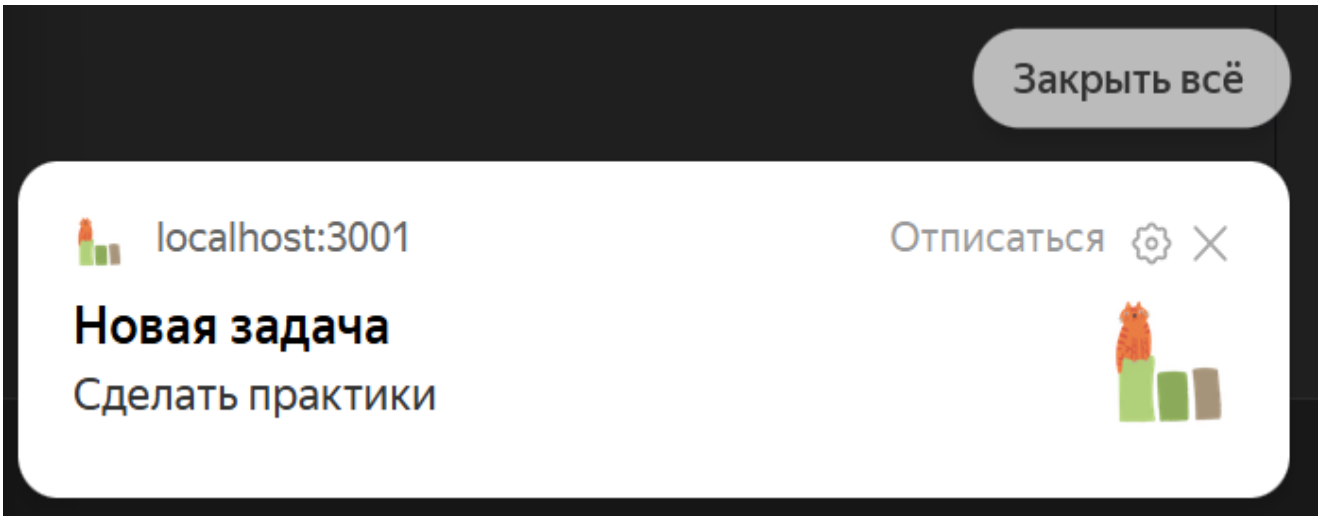
```
socket.on('taskAdded', (task) => {
    console.log('Задача от другого клиента:', task);
    const notification = document.createElement('div');
    notification.textContent = `Новая задача: ${task.text}`;
    notification.style.cssText = `
        position: fixed; top: 10px; right: 10px;
        background: #4285f4; color: white; padding: 1rem;
        border-radius: 5px; z-index: 1000;
    `;
    document.body.appendChild(notification);
    setTimeout(() => notification.remove(), 3000);
});
```

2.3. Модификация Service Worker

В файле `sw.js` добавим обработчик события `push`, который будет показывать системное уведомление:

```
self.addEventListener('push', (event) => {
    let data = { title: 'Новое уведомление', body: '' };
    if (event.data) {
        data = event.data.json();
    }
    const options = {
        body: data.body,
        icon: '/icons/favicon-128x128.png',
        badge: '/icons/favicon-48x48.png'
    };
    event.waitUntil(
        self.registration.showNotification(data.title, options)
    );
});
```

Убедитесь, что иконки по указанным путям существуют (они были добавлены в практической работе №14). При корректном подключении уведомление примет вид:



Шаг 3. Запуск и проверка

1. Запустите сервер командой:

```
node server.js
```

2. Откройте браузер по адресу `http://localhost:3001`. Приложение должно загрузиться.
3. Нажмите кнопку **«Включить уведомления»** и разрешите уведомления, если браузер запросит.
4. Откройте вторую вкладку с тем же адресом.
5. В первой вкладке добавьте новую задачу. Во второй вкладке сразу появится всплывающее сообщение (WebSocket), а также должно прийти системное push-уведомление (если вкладка неактивна, оно отобразится операционной системой).
6. Попробуйте отключить уведомления кнопкой **«Отключить уведомления»** и повторить добавление. Push-уведомления приходить не должны, а WebSocket-уведомления продолжат работать.

Практическое задание

Доработайте веб-приложение для управления списком задач (заметок) следующим образом:

1. Установите необходимые зависимости и реализуйте серверную часть. Сгенерируйте собственные VAPID-ключи и подставьте их в код.
2. Интегрируйте в клиентскую часть библиотеку Socket.IO и реализуйте:
   - подключение к серверу;
   - отправку события `newTask` при добавлении новой задачи;
   - показ всплывающего сообщения при получении события `taskAdded` от сервера.
3. Добавьте возможность подписки на push-уведомления:
   - реализуйте функции `subscribeToPush` и `unsubscribeFromPush` с использованием `PushManager`;

- добавьте в интерфейс кнопки для включения/отключения уведомлений и свяжите их с этими функциями;
- отправляйте подписку на сервер (эндпоинт `/subscribe`) и удаляйте её при отписке (`/unsubscribe`).

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.